



Advanced Multi-Source RAG System

For Enterprise Knowledge Base — An end-to-end AI application that answers questions from PDFs, CSV files, and websites using Retrieval-Augmented Generation.

AI / NLP / RAG

STREAMLIT · GROQ · RENDER

PREPARED BY UWAIS MUHAMMED KP · 2026



The Problem

Users work with information scattered across **PDFs, CSV files, and websites**. Standard search requires manual reading and comparison, while standalone LLMs produce answers **not grounded in the user's own content**.

Fragmented Sources

Multiple formats, no unified query interface

Ungrounded Answers

LLMs hallucinate without source context

Manual Effort

Reading and comparing across sources is slow

Project Objectives

1

Multi-Format Support

Accept PDF, CSV, and website URLs as input sources

2

Smart Retrieval

Select the most relevant content for each user query

3

Grounded Generation

Generate answers using the Groq API with retrieved context

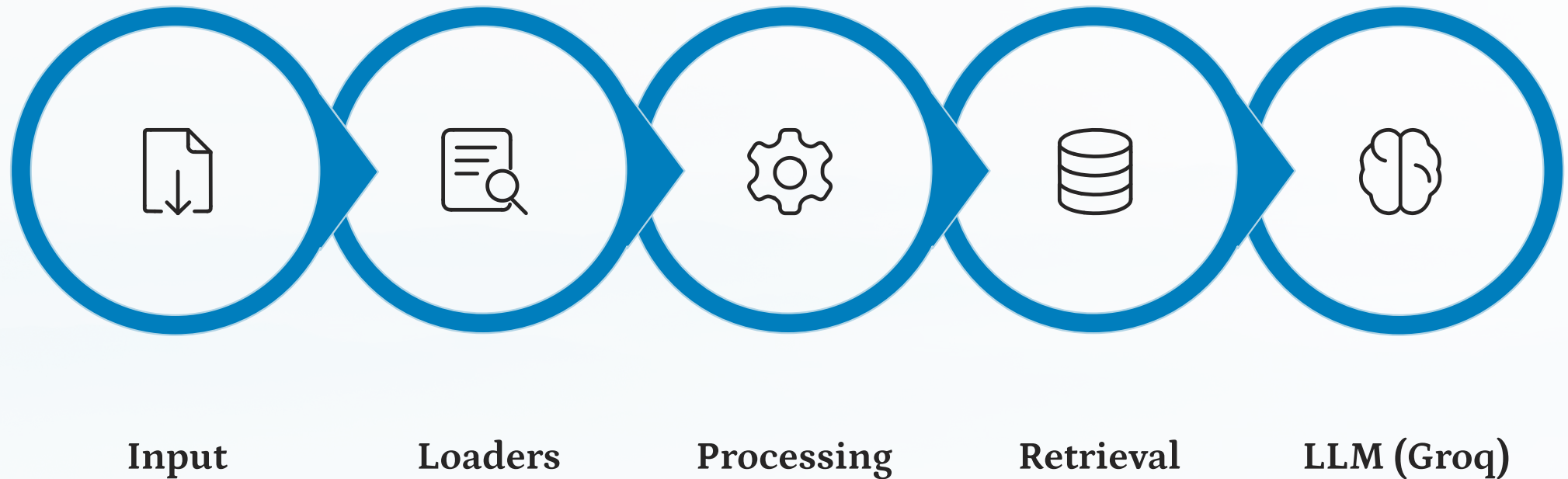
4

Public Deployment

Host live on Render for portfolio demonstration

System Architecture

A modular pipeline where each stage has a focused responsibility — improving readability, extensibility, and debugging.



This design makes each component independently discussable — ideal for interviews and future upgrades.

Core Modules



PDF & CSV Loaders

Reads PDF pages and converts CSV rows into text using PyPDF and pandas



Web Loader

Fetches webpage text, stripping scripts and styles via requests + BeautifulSoup



Cleaning & Chunking

Normalizes formatting and divides content into overlapping, manageable chunks



Retrieval & Response

Matches query to relevant chunks, then prompts Groq to generate the final answer

Implementation Stack



Tool	Purpose
Python	Core language
Streamlit	Interactive UI
PyPDF / pandas	PDF & CSV extraction
requests + BeautifulSoup up	Website ingestion
Groq API	Fast LLM inference
Render	Live cloud hosting

A virtual environment isolated dependencies. Debugging included resolving package conflicts, tuning retrieval logic, and configuring cloud environment variables.

Deployment on Render

The completed project was pushed to **GitHub** and deployed live on **Render**. A custom start script ensured Streamlit ran on the cloud-assigned port, and environment variables securely stored the Groq API key.

GitHub

Version-controlled source repository

Render

Live, publicly accessible web application

Env Vars

Secure API key configuration



Testing & Results

✓ Observed Strengths

- Multiple input formats in one unified app
- Clear end-to-end pipeline from source to answer
- Fast answer generation once context is selected
- Live deployment improves credibility and accessibility
- Modular code supports future upgrades

⚠ Observed Limitations

- Retrieval logic is simpler than production-grade vector search
- Some websites restrict access or return partial content
- CSV analysis is text-based, not deeply analytical
- No long conversation history maintained yet

Challenges & Future Scope

Key Challenges

- **Dependency management** — package conflicts when integrating advanced retrieval libraries
- **Web scraping** — some sites block access or return incomplete text
- **Cloud config** — careful environment variable setup required for Render

Future Enhancements

- Vector database integration (FAISS / ChromaDB)
- Citation-style source references and highlighting
- Conversational memory and chat history
- Improved CSV reasoning with filtering & aggregation
- Support for DOCX and PPTX formats
- Enhanced UI with chat layout and loading indicators



Conclusion

The **Advanced Multi-Source RAG System** demonstrates how retrieval and generation can be combined into a practical, deployed AI application — supporting multiple source types, an interactive interface, and a live public URL.

Academic Value

Strong project for academic presentation and portfolio use

Skills Demonstrated

Programming, AI integration, retrieval logic, UI, and cloud deployment

End-to-End Workflow

One complete pipeline — from raw source to grounded answer